

AURIKREX BYTES: PUSH NOTIFICATION SYSTEM

Comprehensive Technical Architecture, Infrastructure Blueprint & Operational Guide

Target System: Aurikrex Bytes PWA	Document Version: 2.0 (Production)
Architecture Pattern: Dual Web Push & OneSignal SDK	Publication Date: September 16, 2026
Supported Platforms: Android, iOS (PWA), Desktop	Automation Engine: GitHub Actions + Vercel Cron

1. Executive Summary

The Aurikrex Bytes Push Notification System is engineered to deliver timely, reliable, cross-platform notifications directly to user devices—even when the website tab is completely closed or running in the background. The system utilizes a high-availability **dual-layer notification strategy** combining native **W3C Web Push (VAPID RFC 8292)** with the **OneSignal Web SDK**.

Notifications are dispatched on a strict daily schedule (8:01 AM, 10:00 PM, and 12:00 AM WAT) to keep readers engaged with daily tech bytes, maintain reading streaks, and broadcast automated post updates.

KEY ARCHITECTURAL HIGHLIGHTS:

- **Background Delivery:** Notifications land directly in device lock screens & status bars via background Service Workers.
- **Zero Single-Point-of-Failure:** Dual broadcast paths ensure delivery via native Web Push even if third-party SDKs are blocked.
- **Automated Cron Engine:** GitHub Actions bypasses Vercel Hobby plan single-cron restrictions to support multi-time daily pushes.
- **End-to-End Encryption:** Payloads encrypted using elliptic curve ECDH (P-256) and AES-128-GCM.

2. System Architecture & Component Diagram

The overall system spans client browsers, service workers, database persistence, serverless backend functions, and external GitHub Actions cron schedulers:

```
+-----+
| CLIENT LAYER (BROWSER / PWA) |
| +-----+ +-----+ |
| | PushSubscribeButton.tsx | | Service Worker (/sw.js) | |
| | Request Notification Perm | | - Listens for 'push' events | |
| | Obtains PushSubscription | | - Executes showNotification() | |
| +-----+ +-----+ |
+-----+-----^-----+
| tRPC Mutation | Encrypted WebPush
v |
+-----+-----+-----+
| SERVER BACKEND (VERCEL / NODE) |
| +-----+ +-----+ |
| | SQLite / Drizzle ORM | | server/push.ts | | |
| | push_subscriptions table | <----- | - VAPID RSA Signing | |
| | (endpoint, p256dh, auth) | | Query Subs | - webpush.sendNotification() | |
| +-----+ +-----+ | - OneSignal REST API Broadcast | |
| +-----+ +-----+ |
+-----+-----^-----+
| HTTP GET /api/cron/notify
+-----+-----+-----+
| SCHEDULER & AUTOMATION LAYER |
| +-----+ +-----+ |
| | GitHub Actions Workflow (.github/workflows/daily-cron.yml) | |
| | Schedules: 8:01 AM WAT (07:01 UTC) | 10:00 PM WAT (21:00 UTC) | 12:00 AM WAT | |
| +-----+ +-----+ |
+-----+-----+-----+
```

3. Deep-Dive Component Specifications

3.1 Client-Side Subscription & Handshake (PushSubscribeButton.tsx)

When the user clicks the **Alerts bell button** in the header or dashboard, the frontend initiates a multi-stage permission protocol:

- 1. **Feature & OS Support Check:** Verifies `navigator.serviceWorker`, `window.PushManager`, and `window.Notification`. On iOS Safari outside PWA mode, alerts the user to add the app to Home Screen.
- 2. **User Permission Prompt:** Triggers `Notification.requestPermission()`. If granted, proceeds to token registration.
- 3. **Service Worker Registration:** Ensures `/sw.js` is registered and active (`navigator.serviceWorker.ready`).
- 4. **VAPID Key Conversion:** Converts the server's VAPID public key from Base64 URL-safe format to a `Uint8Array` byte array using `urlBase64ToUint8Array()`.
- 5. **Push Subscription Generation:** Invokes `reg.pushManager.subscribe({ userVisibleOnly: true, applicationServerKey })` to negotiate push credentials with browser push servers (Google FCM, Apple Push Services, Mozilla Push).
- 6. **Database Synchronization:** Sends the resulting `endpoint`, `p256dh` public key, and `auth` secret to the server via the tRPC mutation `reader.subscribePush`.

3.2 Service Worker Event Handler (client/public/sw.js)

The Service Worker operates independently of active browser tabs, remaining active in the OS background:

- **OneSignal SDK Worker Import:** Attempts `importScripts('https://cdn.onesignal.com/sdks/web/v16/OneSignalSDK.sw.js')` with fail-safe try-catch wrapper.
- **Push Event Listener:** Intercepts incoming push payloads, parses JSON containing `title`, `body`, and `url`, and triggers `self.registration.showNotification()` with vibration patterns (`[200, 100, 200]`) and custom branding icons.
- **Notification Click Listener:** Closes the banner, checks for existing open windows/tabs of Aurikrex Bytes to focus them, or invokes `self.clients.openWindow(url)` to open the targeted article or dashboard.

3.3 Backend Push Delivery Engine (server/push.ts)

The server backend exposes `sendDailyPushNotifications()`, which runs a dual-path notification dispatch:

```
// 1. OneSignal REST API Broadcast
await fetch('https://onesignal.com/api/v1/notifications', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'Authorization': `Basic ${ONE_SIGNAL_KEY}` },
  body: JSON.stringify({
    app_id: ONE_SIGNAL_APP_ID,
    included_segments: ['Subscribed Users'],
    headings: { en: 'Time for your daily bytes! 📄' },
    contents: { en: 'Catch up on what matters in tech.' },
    url: 'https://www.bytes.aurikrex.tech/dashboard'
  })
});

// 2. Direct Web Push Fallback via Drizzle ORM & web-push
const subs = await db.select().from(pushSubscriptions);
for (const sub of subs) {
  await webpush.sendNotification({ endpoint: sub.endpoint, keys: { p256dh: sub.p256dh, auth: sub.auth } }, payload);
}
```

4. Database Schema & Data Persistence

Push subscriptions are stored in SQLite / Turso using Drizzle ORM in the `push_subscriptions` table:

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique subscription identifier
reader_id	INTEGER	REFERENCES readers(id) ON DELETE CASCADE	Optional associated reader account ID
endpoint	TEXT	NOT NULL, UNIQUE	Browser vendor push service URL
p256dh	TEXT	NOT NULL	Client ECDH public key (P-256 curve)
auth	TEXT	NOT NULL	Client symmetric authentication secret
created_at	INTEGER (timestamp_ms)	NOT NULL, DEFAULT NOW	Subscription registration timestamp

5. Automation, Cron Schedulers & Vercel Constraints

The Vercel Hobby Plan Restriction: Vercel limits free deployments to **1 cron job** executing at most **once per 24 hours** (`schedule: '1 8 \* \* \*')`. Any attempt to add multiple cron expressions in `vercel.json` causes build deployment rejection.

The GitHub Actions Solution: To support multiple daily notifications (8:01 AM, 10:00 PM, and 12:00 AM WAT), the scheduled invocation is offloaded to GitHub Actions (`.github/workflows/daily-cron.yml`):

Trigger Time (WAT)	UTC Schedule	Cron Expression	Target Endpoint & Action
8:01 AM WAT	7:01 AM UTC	1 7 * * *	GET /api/cron/notify (Morning Brief)
10:00 PM WAT	9:00 PM UTC (21:00)	0 21 * * *	GET /api/cron/notify (Nightly Recap)
12:00 AM WAT	11:00 PM UTC (23:00)	0 23 * * *	GET /api/cron/notify (Midnight Test/Brief)
Hourly	Every Hour	0 * * * *	GET /api/cron/publish (Auto-publish scheduled posts)

6. Cross-Platform OS & Browser Compatibility

Push notification behavior varies by operating system and browser context:

Platform / OS	Browser Support	Background Push Support	Special Requirements / Operational Notes
Android	Chrome, Edge, Firefox, Brave	FULL (Native OS Banners)	Works natively out of the box in mobile browser and installed PWA.
Windows Desktop	Chrome, Edge, Firefox	FULL (Action Center)	Delivers system tray toasts & Windows Action Center notifications.
macOS Desktop	Safari 16+, Chrome, Firefox	FULL (Notification Center)	Requires system notification permissions allowed in macOS Settings.
iOS / iPadOS	Safari 16.4+, Chrome (iOS 16.4+)	REQUIRES PWA HOME SCREEN	User MUST tap Share -> 'Add to Home Screen' and launch PWA from Home Screen to receive Web Push on iOS.

7. Operations & Troubleshooting Playbook

• Manual Test Execution via Terminal:

To trigger an immediate push broadcast across all active devices without waiting for cron timing, run:

```
curl.exe -s -X GET "https://www.bytes.aurikrex.tech/api/cron/notify" -H "x-vercel-cron: 1"
```

• Diagnostic Flow for Non-Receiving Users:

1. Check browser permission: Verify address bar settings show Notification permission as 'Allowed'.
2. Verify Service Worker: Inspect Chrome `chrome://serviceworker-internals` or DevTools -> Application -> Service Workers to confirm `sw.js` is active.
3. Test resync: Click the ****Alerts**** button on Aurikrex Bytes to re-trigger subscription registration and refresh push tokens.
4. Check Database count: Verify backend log output `[Push] Found X push subscriptions in database.`